

DVC Deep Dive

Introduction

How to use DVC to version datasets and pipeline artifacts, reproduce outputs, compare changes, and (optionally) sync data through a remote *without* committing large files into Git.

Prereqs (tools)

- Python 3.10+ and a virtual environment (recommended)
- git installed
- dvc installed (pip or system install)

1 Setup: repo + virtual environment (5 minutes)

1. Create a fresh folder and enter it:

```
mkdir dvc-deepdive-demo
cd dvc-deepdive-demo
```

2. Create and activate a venv:

macOS/Linux

```
python -m venv .venv
source .venv/bin/activate
```

Windows PowerShell

```
python -m venv .venv
.venv\Scripts\Activate.ps1
```

3. Install DVC and data libs:

```
pip install dvc pandas scikit-learn
```

4. Initialize Git and DVC:

```
git init
dvc init
git add .
git commit -m "Init git + dvc"
```

2 Core concept refresher

- Git tracks **code** + **tiny pointer files** (*.dvc, dvc.yaml, params.yaml).
- DVC tracks **data** + **pipeline outputs** in a content-addressed cache (hash-based).
- Reproducibility = Git commit + DVC-tracked data versions.

3 Part A: Basic data versioning with dvc add

3.1 A1. Create a small raw dataset

1. Create folders:

```
mkdir -p data/raw data/processed src
```

2. Create a sample CSV at data/raw/customers.csv:

```
customer_id,age,income,churned
1,25,42000,0
2,40,78000,1
3,19,25000,0
4,55,90000,0
5,33,61000,1
```

3.2 A2. Track the raw dataset using DVC

1. Track the dataset (this creates a pointer file):

```
dvc add data/raw/customers.csv
```

2. See what appeared and try to understand:

- data/raw/customers.csv.dvc (pointer tracked by Git)
- data/.gitignore updated so Git ignores data/raw/customers.csv

3. Stage and commit the pointer + gitignore:

```
git add data/raw/customers.csv.dvc data/raw/.gitignore
git commit -m "Track raw dataset with DVC"
```

4. Show status (should be up-to-date):

```
dvc status
```

3.3 A3. Show history of a dataset

1. Show Git history for the dataset pointer:

```
git log --oneline data/raw/customers.csv.dvc
```

2. To remember:

- Each commit of *.dvc corresponds to a **data version**.

3.4 A4. Modify data and record a new version (the “right way”)

1. Edit `data/raw/customers.csv` (e.g., add a new row, or change income).
2. Show DVC detects it:

```
dvc status
```

3. Record the new version:

```
dvc add data/raw/customers.csv
git add data/raw/customers.csv.dvc
git commit -m "Raw dataset v2"
```

4. Show history now has two versions:

```
git log --oneline -- data/raw/customers.csv.dvc
```

3.5 A5. Roll back to an older dataset version (checkout)

1. Identify the older commit hash from:

```
git log --oneline -- data/raw/customers.csv.dvc
```

2. Checkout the older commit:

```
git checkout <OLD_COMMIT_HASH>
```

3. Restore the corresponding data file from DVC cache:

```
dvc checkout
```

4. Verify the file content changed back.
5. Return to latest:

```
git checkout main
dvc checkout
```

4 Part B: Move from “files” to “pipelines” with `dvc.yaml` (15 minutes)

4.1 B1. Create a tiny pipeline stage

Goal: raw → processed with reproducible commands.

1. Create `src/make_processed.py`:

```
import pandas as pd
from pathlib import Path

RAW = Path("data/raw/customers.csv")
OUT = Path("data/processed/customers_clean.csv")

df = pd.read_csv(RAW)
df = df.dropna(subset=["age", "income"])
df = df[df["age"].between(18, 90)]
OUT.parent.mkdir(parents=True, exist_ok=True)
df.to_csv(OUT, index=False)
print("Wrote:", OUT)
```

2. Create a DVC stage (writes `dvc.yaml`):

```
dvc stage add -n process_data \
  -d src/make_processed.py -d data/raw/customers.csv \
  -o data/processed/customers_clean.csv \
  python src/make_processed.py
```

`dvc stage add` -> adding a new pipeline

`-n process_data` -> names the pipeline `process_data`

`-d filename` -> if any of these dependency file changes, the pipeline is outdated and must be re-run

`-o output` writes the output

`python src/*.py` -> dvc runs this command for every run of the pipeline.

3. Run the pipeline:

```
dvc repro
```

4. Commit pipeline definition:

```
git add dvc.yaml dvc.lock src/make_processed.py
git commit -m "Add processing stage (dvc.yaml)"
```

If someone wants to reproduce everything, they check out the Git commit, which restores the exact `dvc.yaml`, `dvc.lock`, and code files. DVC then uses the hashes stored in `dvc.lock` to retrieve the exact versions of the data from cache or remote storage, or to verify that recomputation would produce the same outputs. Whats wrong with `.dvc` then?

`.dvc` files version individual data artifacts, while `dvc.lock` versions the entire pipeline state. You need `dvc.lock` to guarantee end-to-end reproducibility, not just a single file.

4.2 B2. What `dvc repro` gives you

- Incremental builds: reruns stages only when dependencies change.
- Reproducibility: `dvc.lock` locks exact inputs/outputs (hashes).

4.3 B3. Show reproducibility behavior

1. Run again (should say it's up to date):

```
dvc repro
```

2. Modify raw data slightly, then rerun:

```
dvc repro
```

3. Explain:

- DVC detects dependency changes and rebuilds only what is needed.

5 Part C: Params, metrics, and diffs

5.1 C1. Add a parameter file params.yaml

1. Create params.yaml:

```
cleaning:
  min_age: 18
  max_age: 90
```

2. Update src/make_processed.py to read params:

```
import pandas as pd
import yaml
from pathlib import Path

RAW = Path("data/raw/customers.csv")
OUT = Path("data/processed/customers_clean.csv")
PARAMS = Path("params.yaml")

params = yaml.safe_load(PARAMS.read_text())
min_age = params["cleaning"]["min_age"]
max_age = params["cleaning"]["max_age"]

df = pd.read_csv(RAW)
df = df.dropna(subset=["age", "income"])
df = df[df["age"].between(min_age, max_age)]
OUT.parent.mkdir(parents=True, exist_ok=True)
df.to_csv(OUT, index=False)
print("Wrote:", OUT)
```

3. Install PyYAML:

```
pip install pyyaml
```

4. Tell DVC the stage depends on params:

```
dvc stage add -f -n process_data \  
-d src/make_processed.py -d data/raw/customers.csv -d params.yaml \  
-o data/processed/customers_clean.csv \  
python src/make_processed.py
```

5. Run:

```
dvc repro
```

6. Commit:

```
git add params.yaml dvc.yaml dvc.lock src/make_processed.py  
git commit -m "Add params.yaml and param-driven processing"
```

5.2 C2. Add a simple “metric” (validation report)

A metric is a small report file you can diff between versions.

1. Create `src/validate.py` that writes `reports/validation.txt`:

```
import pandas as pd  
from pathlib import Path  
  
DATA = Path("data/processed/customers_clean.csv")  
OUT = Path("reports/validation.txt")  
OUT.parent.mkdir(parents=True, exist_ok=True)  
  
df = pd.read_csv(DATA)  
  
lines = []  
lines.append(f"rows={len(df)}")  
lines.append(f"cols={len(df.columns)}")  
lines.append(f"age_min={df['age'].min()}")  
lines.append(f"age_max={df['age'].max()}")  
lines.append(f"income_min={df['income'].min()}")  
lines.append(f"income_max={df['income'].max()}")  
  
OUT.write_text("\n".join(lines) + "\n")  
print("Wrote:", OUT)
```

2. Add a second DVC stage that depends on processed data and outputs a report:

```
dvc stage add -n validate \  
-d src/validate.py -d data/processed/customers_clean.csv \  
-o reports/validation.txt \  
python src/validate.py
```

3. Run pipeline:

```
dvc repro  
validate.py
```

4. Commit:

```
git add dvc.yaml dvc.lock src/validate.py  
git commit -m "Add validation stage and report output"
```

6 Part D: Day-to-day DVC practices (what you should memorize)

Most-used commands

<code>dvc init</code>	<code># enable DVC in a git repo</code>
<code>dvc add <file/dir></code>	<code># start tracking data file/dir</code>
<code>dvc status</code>	<code># see what changed (DVC-tracked)</code>
<code>dvc checkout</code>	<code># restore workspace to match current git commit</code>
<code>dvc stage add ...</code>	<code># define pipeline stage in dvc.yaml</code>
<code>dvc repro</code>	<code># run pipeline with dependency tracking</code>
<code>dvc diff <A> </code>	<code># compare outputs between versions/commits</code>

Best practices

- Always commit `dvc.yaml` and `dvc.lock` along with code changes.
- Never commit large datasets to Git; let DVC manage them.
- Use `params.yaml` for knobs you want to tune and reproduce.
- Use small text reports as metrics (validation summaries, counts, etc.).